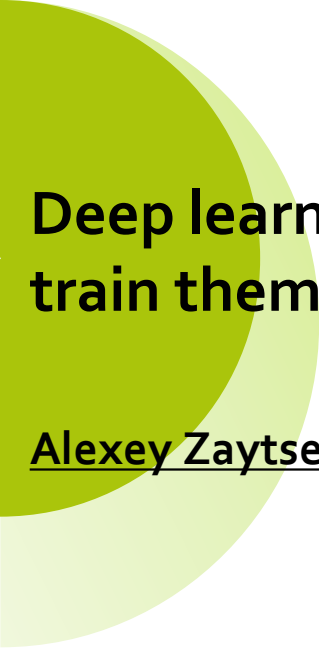




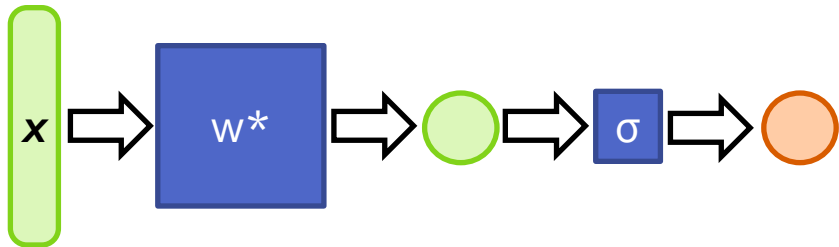
Deep learning models & how to train them

Alexey Zaytsev, Evgeny Burnaev



Logistic regression recap

Logistic regression recap

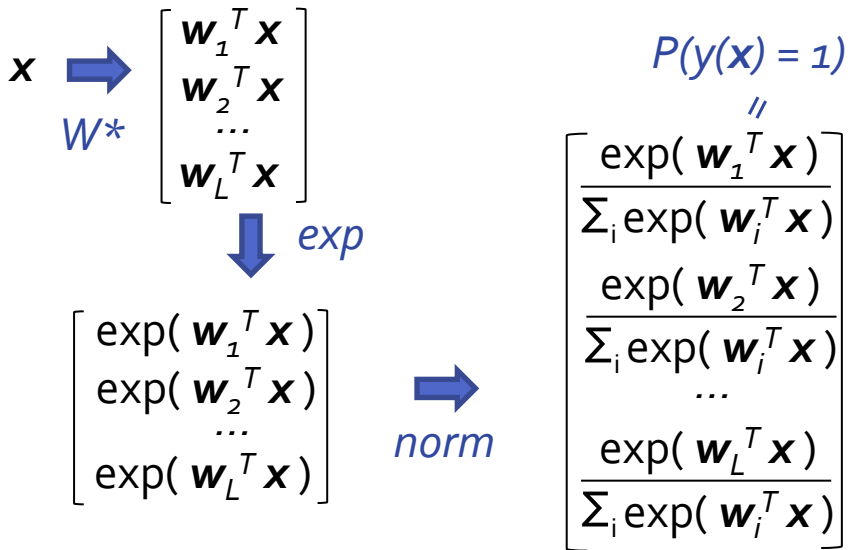


$$P(y(\mathbf{x}_i) = y_i | \mathbf{w}) = \frac{1}{1 + \exp(-y_i \mathbf{w}^T \mathbf{x}_i)} = \sigma(y_i \mathbf{w}^T \mathbf{x}_i)$$

The loss function for the logistic regression

$$E(\mathbf{w}) = - \sum_{i=1} \log P(y(\mathbf{x}_i) = y_i | \mathbf{w})$$

Softmax: a sigmoid generalization for multiclass classification



Multinomial logistic loss

$$P(y(x) = i) = \frac{\exp w_i^T x}{\sum_j \exp w_j^T x}$$

Multinomial log loss generalizes the logistic loss:

$$\begin{aligned} E(w) &= - \sum_i \log P(y(x_i) = y_i) = \\ &= - \sum_i \left[w_{y_i}^T x_i - \log \sum_j \exp w_j^T x_i \right] \end{aligned}$$

(Part of the) gradient over $\mathbf{w}_j$:

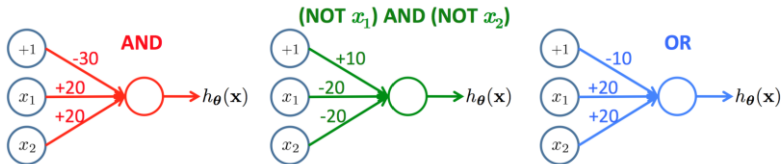
$$\frac{dE}{dw_j} = - \sum_i x_i ([y_i == j] - P(y(x_i) = j))$$

Few notes on logistic regression

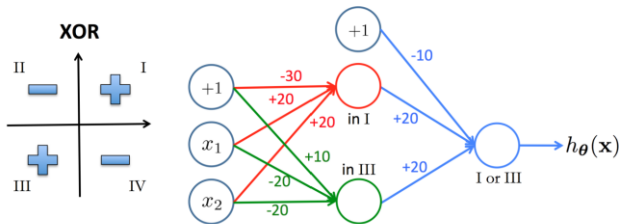
- Convex optimization problem.
Exercise: check that Hessian (the matrix of second derivatives) is positive definite
- Requires regularization for parameters:
on by default in sklearn
- Admits multiclass classification
- Logistic regression is a generalized linear model (GLM) & a one-layer neural network

Capabilities are limited: need more layers

Building blocks:



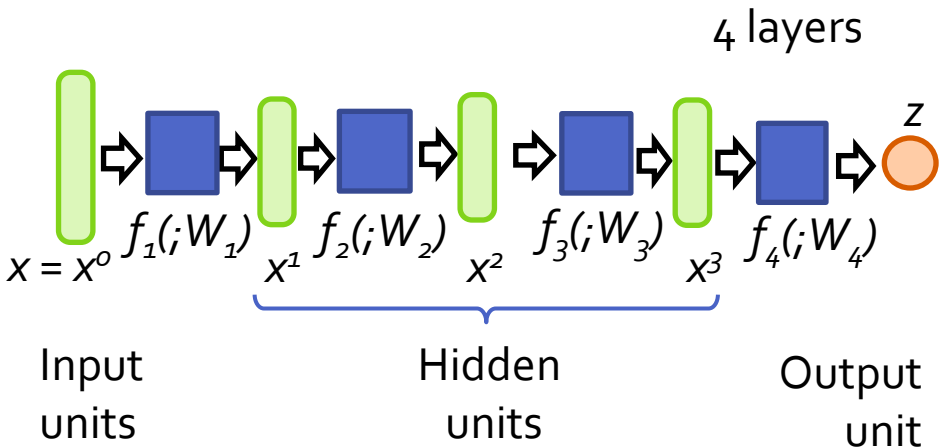
XOR function:



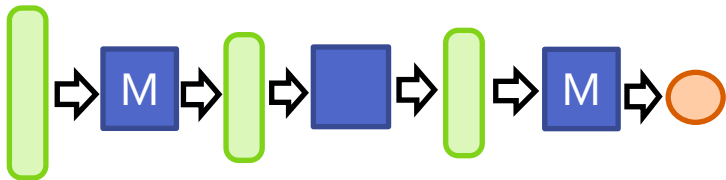


Fully connected neural networks

Multilayer fully-connected (FC) neural network



Universal approximation theorem (UAT)



UAT [Tsybenko, 1989]: given non-polynomial non-linearity, a **single hidden layer** neural network can approximate any continuous function on any compact subset of $\mathbb{R}^d$ up to an arbitrary precision.

Caveat 1: the width of the network can be a very quickly growing function of space dimension and approximation accuracy. Deeper architectures are exponentially narrower for some classes of functions [Rollnick&Tegmark 2018]

Caveat 2: no guarantees on extrapolation beyond the compact set where the approximation is computed. Thus, a proper parameterization is needed.

2006

CAPTCHA



Computer vision = 60%

$$0.6^{12} = 0.00217$$

2014



Completed • Swag • 215 teams

Dogs vs. Cats

Wed 25 Sep 2013 – Sat 1 Feb 2014 (8 months ago)

Dashboard

Private Leaderboard - Dogs vs. Cats

This competition has completed. This leaderboard reflects the final standings.

See someone's

#	Δ1w	Team Name * in the money	Score ?	Entries	Last Submission UTC (Best - Last)
1	—	Pierre Sermanet *	0.98914	5	Sat, 01 Feb 2014 21:43:19 (-)
2	↑26	orchid *	0.98309	17	Sat, 01 Feb 2014 23:52:30
3	—	Owen	0.98171	15	Sat, 01 Feb 2014 17:04:40 (-)
4	new	Paul Covington	0.98171	3	Sat, 01 Feb 2014 23:05:20
5	↓3	Maxim Milakov	0.98137	24	Sat, 01 Feb 2014 18:20:58

$$0.989^{12} = 0.875$$

2014

Microsoft Research

[Our research](#)[Connections](#)[Careers](#)[About us](#)[All](#)[Downloads](#)[Events](#)[Groups](#)[News](#)[People](#)[Projects](#)[Publications](#)

ASIRRA



After 8 years of operation, Asirra is shutting down effective October 1, 2014. Thank you to all of our users!

Classic Machine learning, where representations are available

u – an object

A client

y – true label

Will leave in 3 months?

$x(u)$ – an object
representation

Salary, age

Problem: train a model that
can identify the true class for
 y

A gradient boosting
model

Object



Representation



Classifier



For structured data we need representation learning

u – an object

An image

y – true label

Cat breed? (1)

$x(u)$ – an object representation

NA (2)

Problems: (1) train a model that can identify the true class for y ; **(2) learn a representation**

A neural network:
Encoder +
Classifier

Object



(2) Encoder



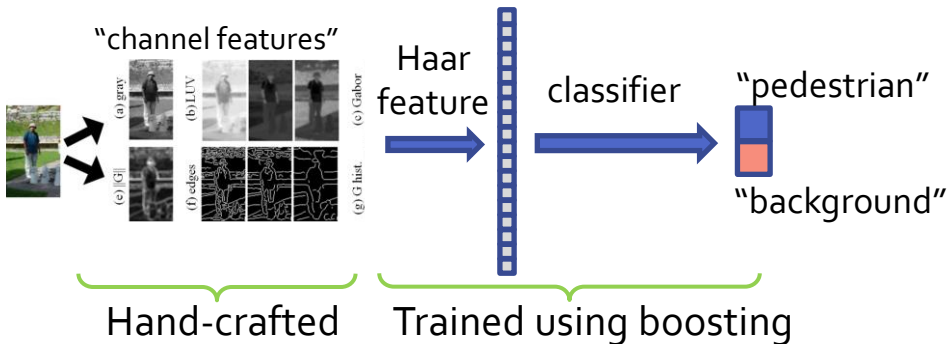
Representation



(1) Classifier



“Deep learning” is not only depth



- Previous CV systems were “deep”, they used multiple layers of representation with certain success
- But they were not called “deep learning”

Deep learning

End-to-end joint learning of all layers:

- multiple assemblable blocks
- each block is piecewise-differentiable
- gradients computed by backpropagation
- gradient-based optimization





Optimization for machine learning recap

Optimization for supervised ML

- $R(\mathbf{w})$ denotes regularization e.g. $||\mathbf{w}'||^2$
- $l(x_i, y_i, \mathbf{w})$ denotes loss for i -th example, e.g. $-\log P(y(\mathbf{x}_i) = y_i | \mathbf{w})$
- The optimization objective is:

$$E(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N l(\mathbf{x}_i, y_i, \mathbf{w}) + \lambda R(\mathbf{w})$$

Small scale setting: traditional optimization

$$E(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N l(\mathbf{x}_i, y_i, \mathbf{w}) + \lambda R(\mathbf{w})$$

- Data are few: we can look through it at each optimization iteration
- Use adapted versions of standard optimization methods including gradient descent, quasi-Newton, quadratic programming, etc.

Gradient descent

$$E(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N l(\mathbf{x}_i, y_i, \mathbf{w}) + \lambda R(\mathbf{w}),$$

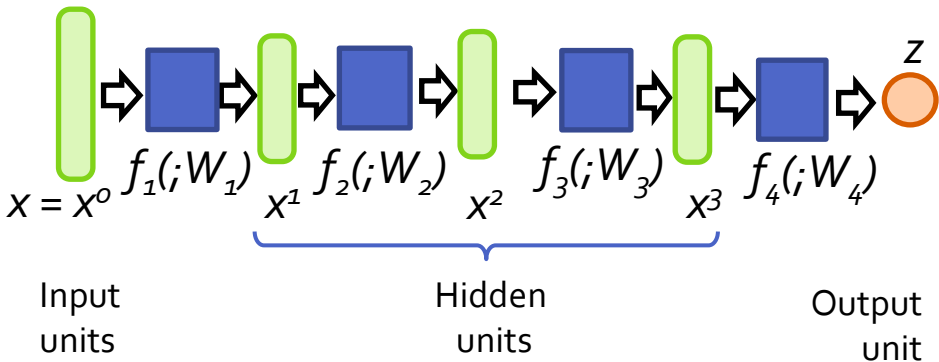
$$\frac{dE(\mathbf{w})}{d\mathbf{w}} = \frac{1}{N} \sum_{i=1}^N \frac{dl(\mathbf{x}_i, y_i, \mathbf{w})}{d\mathbf{w}} + \lambda \frac{dR(\mathbf{w})}{d\mathbf{w}},$$

$$\mathbf{w}^{t+1} = \mathbf{w}^t - \alpha \left. \frac{dE(\mathbf{w})}{d\mathbf{w}} \right|_{\mathbf{w}=\mathbf{w}^t}$$

We need to compute gradients!

For FC models

Backpropagation is a gradient computation algorithm [Shun'ichi]



A. Shun'ichi. A theory of adaptive pattern classifier. *IEEE Transactions*. 1967

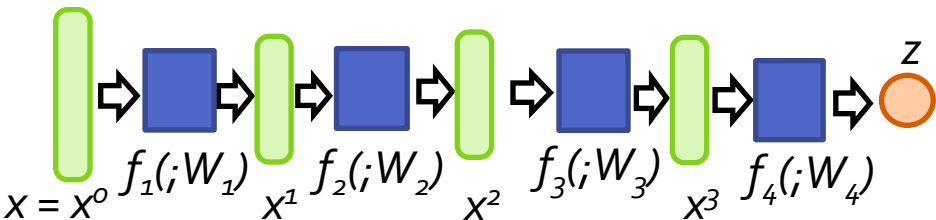
Chain rule for the derivative of the composition of functions

Composition of f and g

$$z = f(g(x))$$

$$\frac{\partial z}{\partial x} = \frac{\partial f(g(x))}{\partial x} = \frac{\partial f(g(x))}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

Chain rule for an FC network



$$z = f_4(x^3; W_4) = f_4(f_3(x^2; W_3); W_4)$$

$$\text{our } f_4(x^3; W_4) = \text{softmax}(W_4 x^3)$$

$$\begin{aligned} \frac{\partial z}{\partial x^2} &= \frac{\partial f_4(x^3; W_4)}{\partial x^2} = \frac{\partial f_4(x^3; W_4)}{\partial x^3} \frac{\partial x^3}{\partial x^2} = \\ &= \frac{\partial f_4(x^3; W_4)}{\partial x^3} \frac{\partial f_3(x^2; W_3)}{\partial x^2} \end{aligned}$$

Sequential computation: *backpropagation*

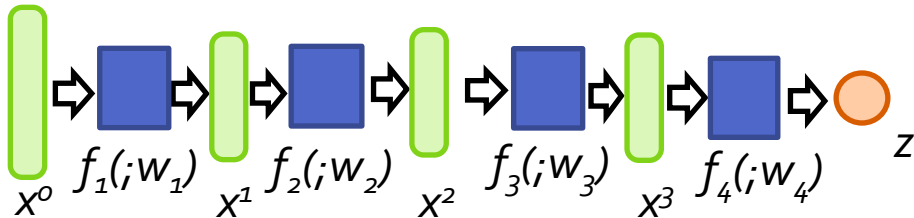
$\frac{dz}{dx^3}, \frac{dz}{dw_4}$ can be computed

$$\frac{dz}{dw_3} = \frac{dx^3}{dw_3}^T \cdot \frac{dz}{dx^3}$$

$$\frac{dz}{dx^2} = \frac{dx^3}{dx^2}^T \cdot \frac{dz}{dx^3}$$

$$\frac{dz}{dw_1} = \frac{dx^1}{dw_1}^T \cdot \frac{dz}{dx^1}$$

$$\frac{dz}{dx^0} = \frac{dx^1}{dx^0}^T \cdot \frac{dz}{dx^1}$$





Optimization for deep learning

Large-scale learning

$$E(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N l(\mathbf{x}_i, y_i, \mathbf{w}) + \lambda R(\mathbf{w}),$$

$$\frac{dE(\mathbf{w})}{d\mathbf{w}} = \frac{1}{N} \sum_{i=1}^N \frac{dl(\mathbf{x}_i, y_i, \mathbf{w})}{d\mathbf{w}} + \lambda \frac{dR(\mathbf{w})}{d\mathbf{w}}$$

Evaluating gradients is very expensive!

Stochastic gradient descent (SGD) idea:

- Evaluate a coarse approximation to grad
- Make “quick” steps

Stochastic gradient descent (SGD)

Gradient:

$$\frac{dE(\mathbf{w})}{d\mathbf{w}} = \frac{1}{N} \sum_{i=1}^N \frac{dl(\mathbf{x}_i, y_i, \mathbf{w})}{d\mathbf{w}} + \lambda \frac{dR(\mathbf{w})}{d\mathbf{w}}$$

Stochastic gradient:

$$\frac{dE^i}{d\mathbf{w}} = \frac{dl(\mathbf{x}_i, y_i, \mathbf{w})}{d\mathbf{w}} + \lambda \frac{dR(\mathbf{w})}{d\mathbf{w}}$$

Stochastic gradient is an unbiased estimate of the gradient:

$$\frac{dE}{d\mathbf{w}} = \frac{1}{N} \sum_{i=1}^N \frac{dE^i}{d\mathbf{w}}$$

Gradient descent (GD)

GD:

$$\begin{aligned}v[t] &= -\alpha[t] \nabla(E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

where $\nabla(E, w[t]) = \frac{dE}{dw}(w[t])$

- $\alpha[t]$ is the learning rate, *more on this later*
- Convergence is guaranteed for good problems (deep learning optimization is not a good problem)

Stochastic gradient descent (SGD)

SGD:

$$\begin{aligned}v[t] &= -\alpha[t] \nabla(E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

where $\nabla(E, w[t]) = \frac{dE^{i(t)}}{dw}(w[t])$

- $i(t)$ usually follow random permutations of training data
- One sweep over training data is called an **epoch**

Stochastic gradient descent (SGD)

SGD:

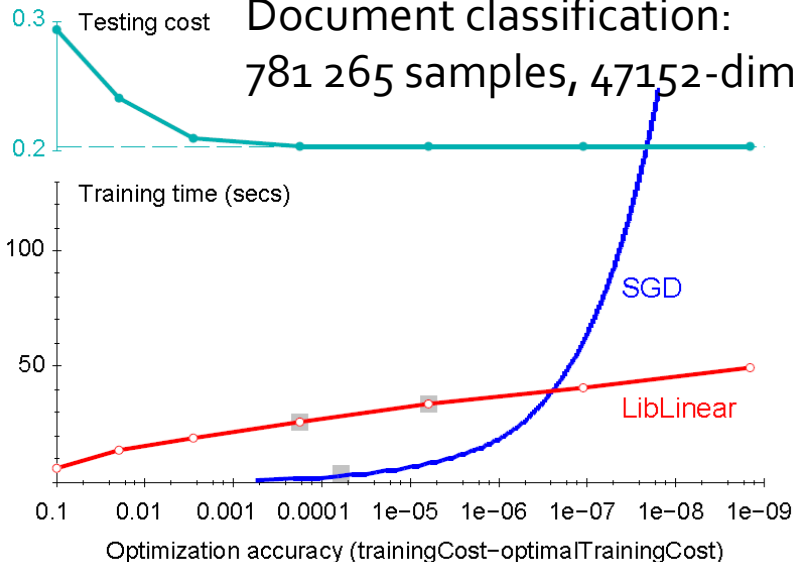
$$\begin{aligned}v[t] &= -\alpha[t] \nabla(E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

- One sweep over training data is called an **epoch**
- Popular choices for schedule $\alpha[t]$:
 - constant, e.g. $\alpha[t] = 0.0001$
 - piecewise constant, e.g. $\alpha[t]$ is decreased tenfold every N epochs
 - harmonic, e.g. $\alpha[t] = 0.001 / ([t/N] + 10)$

The efficiency of SGD ("shallow" learning)

[L.Bottou, 2010]

Document classification:
781 265 samples, 47152-dim BoW



Batch SGD, often called just SGD

Gradient:

$$\frac{dE}{dw} = \frac{1}{N} \sum_{i=1}^N \frac{dl(x_i, y_i, w)}{dw} + \lambda \frac{dR}{dw}$$

Batch is a subset of indexes:

$$B = \{b_1, b_2, \dots, b_{N_b}\} \subset \{1, \dots, N\}$$

Batch stochastic gradient:

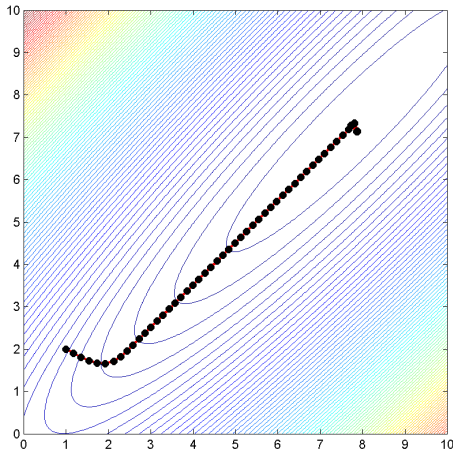
$$\frac{dE}{dw} = \frac{1}{N_b} \sum_{i=1}^{N_b} \frac{dl(x_{b(i)}, y_{b(i)}, w)}{dw} + \lambda \frac{dR}{dw}$$

Why do batching?

$$\frac{dE}{dw} = \frac{1}{N_b} \sum_{i=1}^{N_b} \frac{dl(x_{b(i)}, y_{b(i)}, w)}{dw} + \lambda \frac{dR}{dw}$$

- “Less stochastic” approximation, more stable convergence (questionable)
- **Main reason:** all modern architectures have parallelism, hence computing mini-batch grad is often as cheap as a single stochastic grad

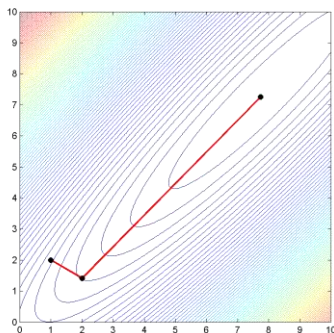
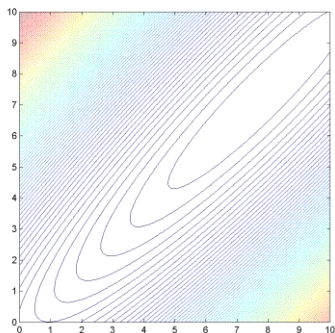
SGD inherits gradient descent problems



- Gradient descent is very poor “in ravines”
- SGD is no better

Better optimization methods

- Second order methods: Newton, Quasi-Newton
- Krylov subspace methods, in particular *conjugate gradients*



Improving SGD using momentum

- Conjugate gradients use a combination of the current gradient and previous direction for the next step
- Similar idea for SGD (*momentum*):

$$\begin{aligned}v[t] &= -\alpha[t] \nabla(E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$



$$\begin{aligned}v[t] &= \mu v[t-1] - \alpha[t] \nabla(E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

Typical $\mu = 0.9$

Exponentially decaying running average

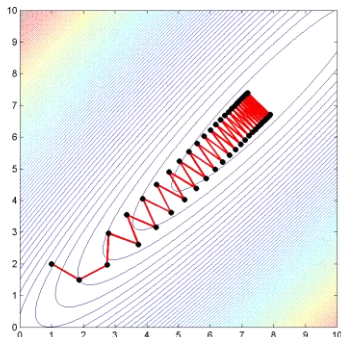
$$\begin{aligned}v[t] &= \mu v[t-1] - \alpha[t] \nabla (E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

$$\begin{aligned}v[t] &= \mu v[t-1] - \alpha[t] \nabla (E, w[t]) = \\&= \mu^2 v[t-2] - \mu \alpha[t-1] \nabla (E, w[t-1]) \\&\quad - \alpha[t] \nabla (f, w[t]) = \\&= \mu^3 v[t-3] - \mu^2 \alpha[t-2] \nabla (E, w[t-2]) \\&\quad - \mu \alpha[t-1] \nabla (E, w[t-1]) - \alpha[t] \nabla (E, w[t]) = \\&= \mu^{k+1} v[t-k-1] + \sum_{i=0}^k \mu^i \alpha[t-i] \nabla (E, w[t-i])\end{aligned}$$

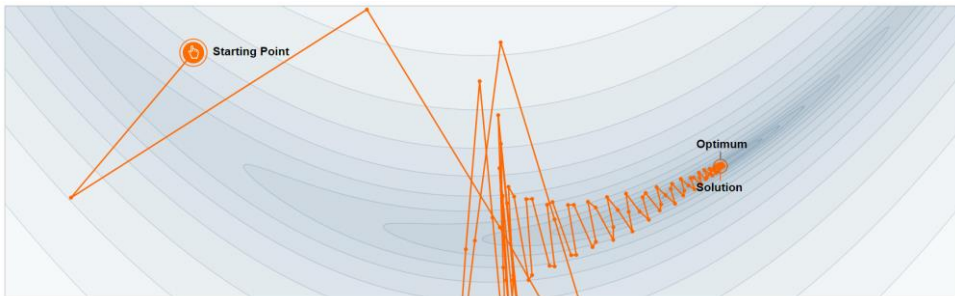
Momentum: why it works

$$v[t] \approx \sum_{i=0}^k \mu^i \alpha[t-i] \nabla(E, w[t-i])$$

- Smoothes out noise in SGD (~bigger batches)
- **Smoothes out oscillations inherent to gradient descent**
- Escapes local minima



The effect of the momentum



Step-size $\alpha = 0.0051$



Momentum $\beta = 0.90$



We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

<https://distill.pub/2017/momentum/>

[Goh, Distill 2017]

Simplified case: quadratic problem

$$E(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T A \mathbf{w} - \mathbf{b}^T \mathbf{w} \rightarrow \min_{\mathbf{w}},$$

A is a symmetric positive-definite matrix with eigenvalues $\{\lambda_1, \dots, \lambda_d\} > 0$

The global optimum:

$$\mathbf{w}^* = A^{-1} \mathbf{b}$$

But let us try to solve this problem with GD

e.g.: this is the problem we solve for linear regression

<https://distill.pub/2017/momentum/> [Goh, Distill 2017]

Gradient descent convergence

$$E(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T A \mathbf{w} - \mathbf{b}^T \mathbf{w} \rightarrow \min,$$

$\mathbf{w}^k$ is the value at iteration k of GD

$$E(\mathbf{w}^k) - E(\mathbf{w}^*) = \sum (1 - \alpha \lambda_i)^{2k} \lambda_i [w_i^0]^2,$$

For convergence we want $(1 - \alpha \lambda_i)^{2k}$ to be smaller than 1.

Optimal convergence rate

$$E(\mathbf{w}^k) - E(\mathbf{w}^*) = \sum (1 - \alpha\lambda_i)^{2k} \lambda_i [w_i^0]^2,$$

The overall convergence rate:

$$\text{rate}(\alpha) = \max\{|1 - \alpha\lambda_1|, |1 - \alpha\lambda_d|\},$$

Optimal convergence rate:

$$\text{rate}(\alpha^*) = \frac{2}{\lambda_1 + \lambda_n}, \alpha^* = \frac{\lambda_n/\lambda_1 - 1}{\lambda_n/\lambda_1 + 1}$$

Simplified case: quadratic problem with momentum

$$\text{rate}(\alpha, \beta) = \max\{|\sigma_1|, |\sigma_2|\},$$

σ_1, σ_2 are eigenvalues of the matrix R:

$$R = (\beta, \lambda_i; -\alpha\beta, 1 - \alpha\lambda_i)$$

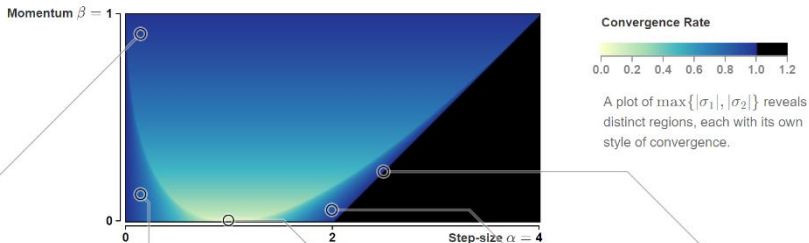
A bit more complex, but e.g. converges is observed for

$$0 < \alpha\lambda_i < 2 + 2\beta, \beta \in [0, 1)$$

<https://distill.pub/2017/momentum/>

[Goh, Distill 2017]

Phase space along a single eigenvector



Ripples



R 's eigenvalues are complex, and the iterates display low frequency ripples. Surprisingly, the convergence rate $2\sqrt{\beta}$ is independent of α and λ_i .

Monotonic Decrease



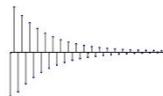
R 's eigenvalues are both real, are positive, and have norm less than one. The behavior here resembles gradient descent.

1-Step Convergence



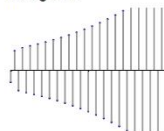
When $\alpha = 1/\lambda_i$, and $\beta = 0$, we converge in one step. This is a very special point, and kills the error in the eigenspace completely.

Monotonic Oscillations



When $\alpha > 1/\lambda_i$, the iterates flip between $+$ and $-$ at each iteration. These are often referred to as 'oscillations' in gradient descent.

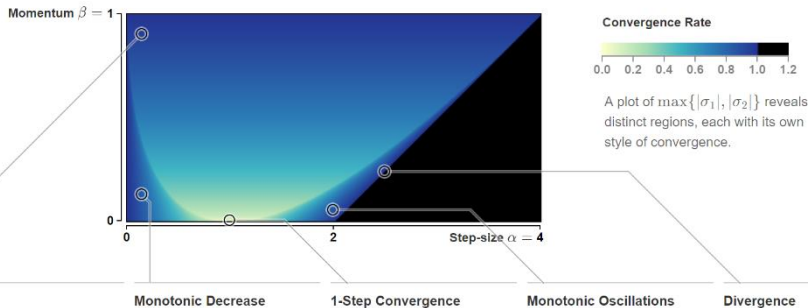
Divergence



When $\max\{|\sigma_1|, |\sigma_2|\} > 1$, the iterates diverge.

[Goh, Distill 2017]

Momentum: multiple eigenvalues



Optimal rate & momentum:

$$\alpha = \left(\frac{2}{\sqrt{\lambda_1} + \sqrt{\lambda_n}} \right)^2 \quad \beta = \left(\frac{\sqrt{\lambda_n} - \sqrt{\lambda_1}}{\sqrt{\lambda_n} + \sqrt{\lambda_1}} \right)^2$$

Optimal speed:

$$\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1}$$

Convergence rate,
Momentum

$$\frac{\kappa - 1}{\kappa + 1}$$

Convergence rate,
Gradient Descent

[Goh, Distill 2017]

Momentum: multiple eigenvalues

Optimal rate &
momentum:

$$\alpha = \left(\frac{2}{\sqrt{\lambda_1} + \sqrt{\lambda_n}} \right)^2 \quad \beta = \left(\frac{\sqrt{\lambda_n} - \sqrt{\lambda_1}}{\sqrt{\lambda_n} + \sqrt{\lambda_1}} \right)^2$$

In real network we do not know eigenvalues and their change with iterations, so:

- we set the momentum high (e.g. 0.9)
- then we tune the learning rate

Nesterov accelerated gradient

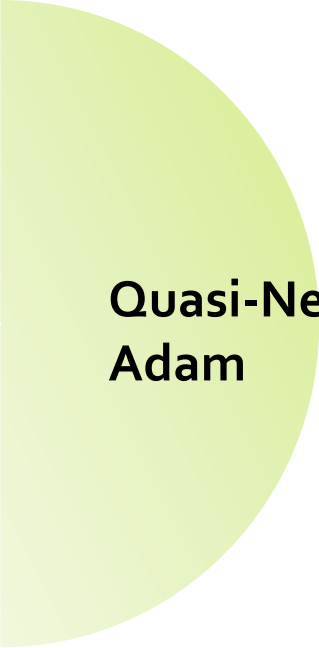
$$\begin{aligned}v[t] &= \mu v[t-1] - \alpha[t] \nabla(E, w[t]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

Before we even compute the gradient, we have a good approximation where we will end up: $w[t+1] \approx w[t] + \mu v[t-1]$

Let us use this knowledge:

$$\begin{aligned}v[t] &= \mu v[t-1] - \alpha[t] \nabla(E, w[t] + \mu v[t-1]) \\w[t+1] &= w[t] + v[t]\end{aligned}$$

(Computing the gradient at a more relevant spot)



Quasi-Newton approach: Adam

Second-order methods

- Exponential smoothing helps, but still not optimal if large anisotropy exists
- Classic (Newton) solution: estimate the Hessian and make the update
$$v[t+1] = -H[t]^{-1} \nabla(E, w[t])$$
 (the lower the curvature the faster we go)
- Quasi-Newton methods: estimate some approximation to Hessian based on observed gradients

Recap from statistics

Let us define the Fisher information matrix

$$I(\mathbf{w}) = -\mathbb{E} \left[\frac{\partial^2}{\partial \mathbf{w}^2} \log f(\mathbf{x}, y | \mathbf{w}) \mid \mathbf{w} \right],$$

where $\mathbb{E}(\mathbf{w}) = f(\mathbf{x}, y | \mathbf{w})$ is the likelihood for a given data point (e.g. scaled squared error)

Under regularity conditions, it holds that

$$I_{ij}(\mathbf{w}) = \mathbb{E} \left[\frac{\partial}{\partial w_i} \log f(\mathbf{x}, y | \mathbf{w}) \frac{\partial}{\partial w_j} \log f(\mathbf{x}, y | \mathbf{w}) \mid \mathbf{w} \right].$$

- The first form is the Hessian for our problem
- We can use sample-based estimates instead
- We can look only at the first derivatives

Adagrad method [Duchi et al. 2011]

Idea: scale updates along different dimensions according to accumulated gradient magnitude

$$g[t] = g[t-1] + \nabla(E, w[t]) \odot \nabla(E, w[t])$$

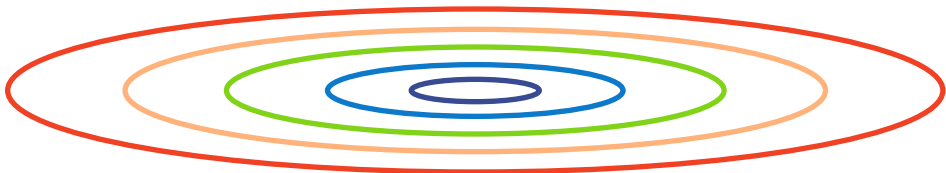
$$w[t+1] = w[t] - \frac{\alpha}{\sqrt{g[t] + \epsilon}} \odot \nabla(E, w[t])$$

Note: step lengths automatically decrease (perhaps too quickly).

Adagrad method [Duchi et al. 2011]

$$g[t] = g[t-1] + \nabla(E, w[t]) \odot \nabla(E, w[t])$$

$$w[t+1] = w[t] - \frac{\alpha}{\sqrt{g[t] + \epsilon}} \odot \nabla(E, w[t])$$



Adagrad in this case: find out that “vertical” derivatives are bigger, then make “vertical” steps smaller than “horizontal”

RMSPROP method [Hinton 2012]

Same as Adagrad, but replace accumulation of squared gradient with the running average:

$$g[t] = \mu g[t-1] + (1-\mu) \nabla(E, w[t]) \odot \nabla(E, w[t])$$
$$w[t+1] = w[t] - \frac{\alpha[t]}{\sqrt{g[t]} + \epsilon} \odot \nabla(E, w[t])$$

Comparison: logistic regression

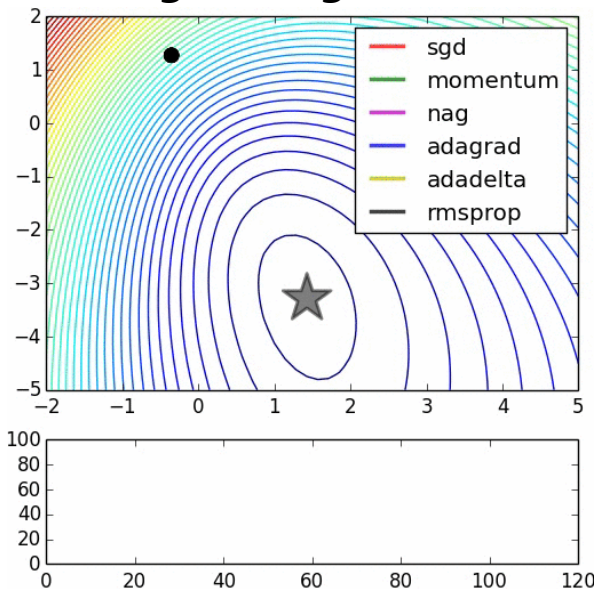


Image credit: Alec Redford

Further comparison

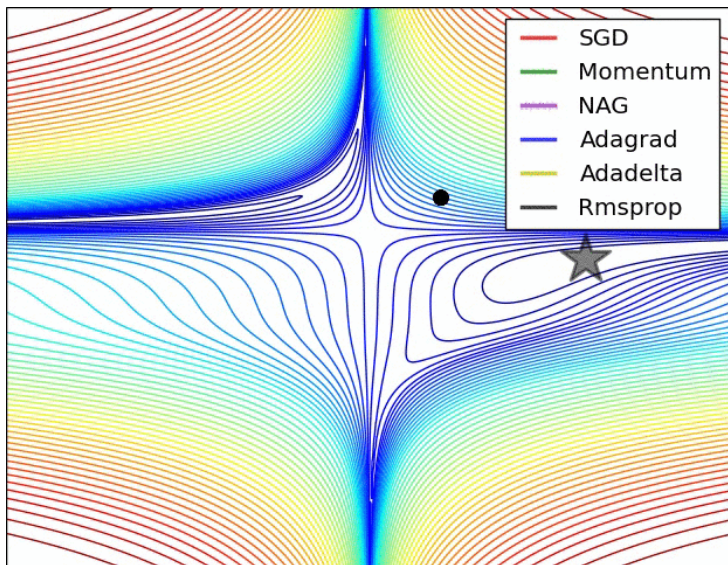


Image credit: Alec Redford

Further comparison: escaping from a saddle

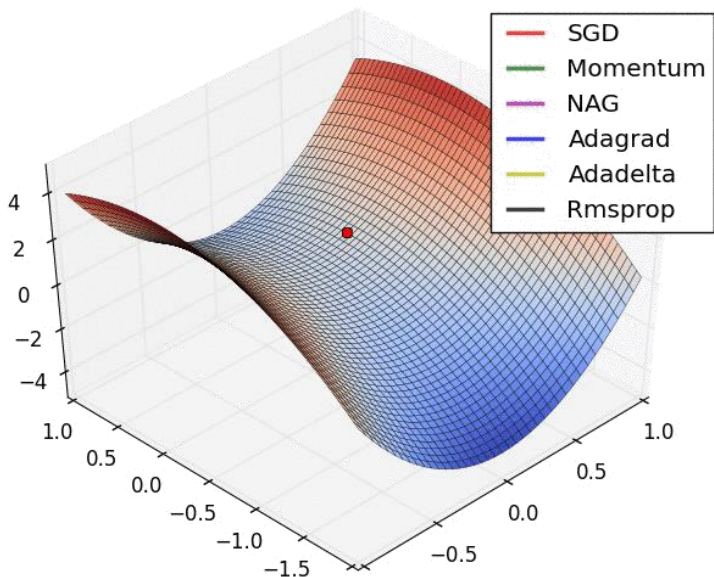


Image credit: Alec Redford

ADAM method [Kingma & Ba 2015]

ADAM = "ADaptive Moment Estimation"

$$v[t] = \beta v[t-1] + (1 - \beta) \nabla(E, w[t])$$

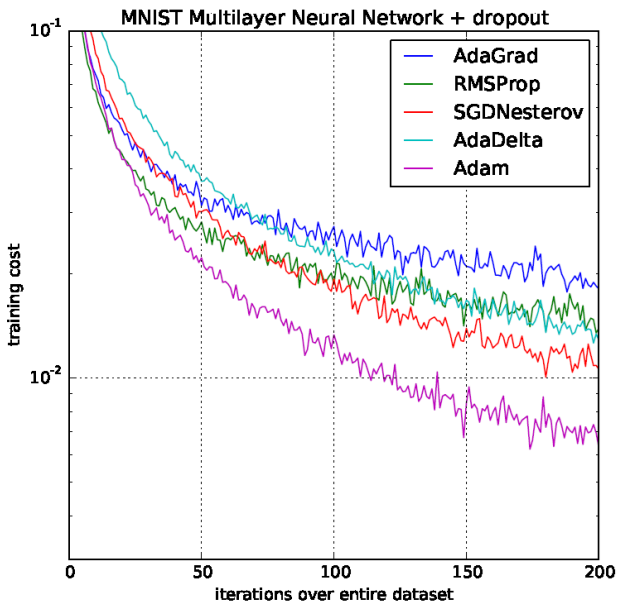
$$g[t] = \mu g[t-1] + (1 - \mu) \nabla(E, w[t]) \odot \nabla(E, w[t])$$

$$w[t+1] = w[t] - \alpha \frac{1}{\sqrt{g[t] + \epsilon}} \odot v[t]$$

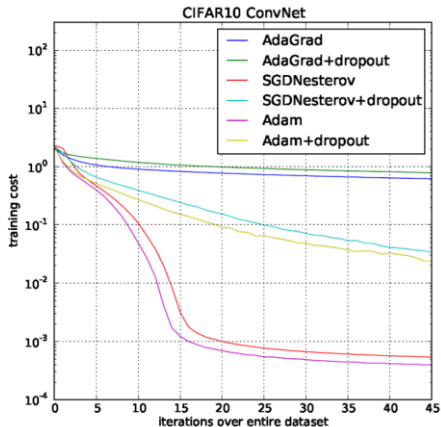
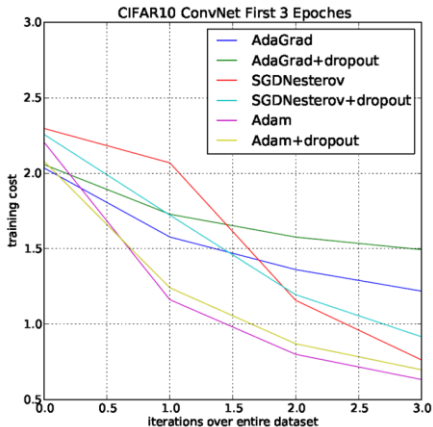
(Note: In the original image, blue lines indicate that the term $\frac{1}{\sqrt{g[t] + \epsilon}}$ is multiplied by $1 - \mu^t$ and the term $v[t]$ is multiplied by $1 - \beta^t$.)

Recommended values: $\beta = 0.9$, $\mu = 0.999$, $\alpha = 0.001$, $\epsilon = 10^{-8}$

ADAM method [Kingma & Ba 2015]

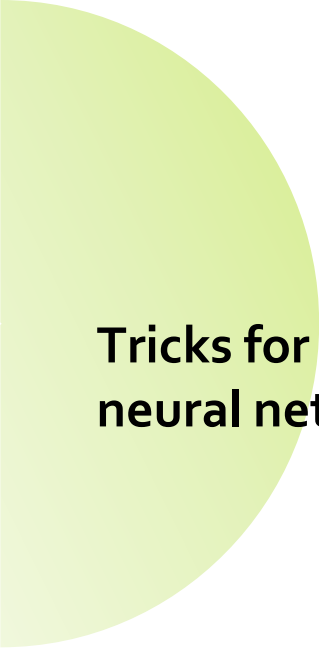


ADAM method [Kingma & Ba 2015]



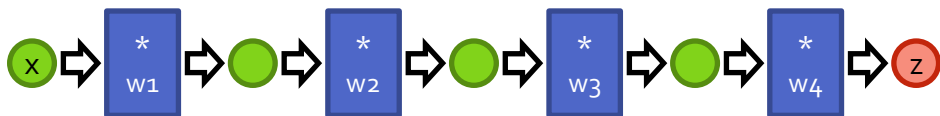
Recap: optimization methods for DL

- Stochastic optimization is used always
- Optimization methods are not trying to estimate full Hessian, ignoring interaction between variables
- ADAM is the most common option
- Also see Nesterov accelerated gradient



Tricks for optimization of neural networks

Problems with DL optimization

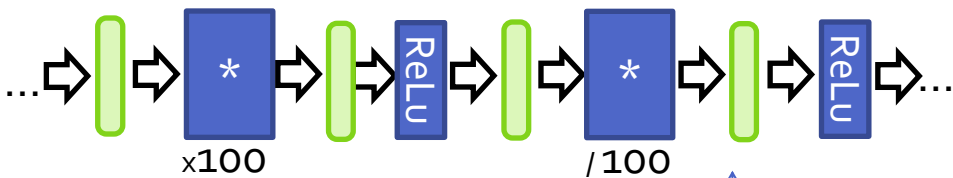


Toy example: $z = w_4 w_3 w_2 w_1 x$

$$\frac{dz}{dw_2} = w_4 w_3 w_1 x \quad \frac{dz}{dw_3} = w_4 w_2 w_1 x$$

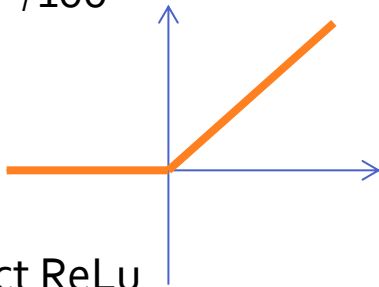
- $w = (1, 1, 1, 1)$ and $w = (1, 0.01, 100, 1)$ define the same function ("gauge freedom"), but very different derivatives
- In the first case, derivatives and values are of order 1.
- In the second case, derivatives and values are wildly different

Gauge freedom in ReLu Networks



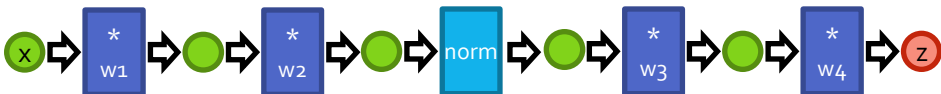
$$\alpha > 0$$

$$1/\alpha \text{ ReLu}(\alpha x) = \text{ReLu}(x)$$



Thus: we can easily construct ReLu networks with **different** weights implementing the **same** function

Normalizing in the toy example



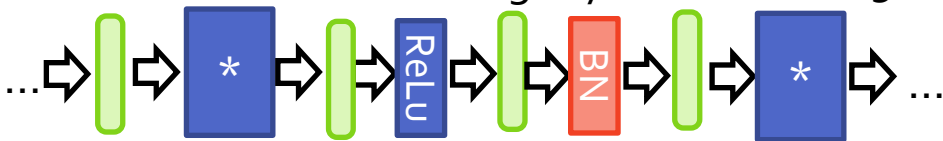
Toy example:
$$z_i = w_4 w_3 \frac{w_2 w_1 x_i}{\frac{1}{N} \sum_{j=1}^N w_2 w_1 x_j}$$

$$\left[\frac{dz}{dw_3} \right]^i = w_4 \frac{w_2 w_1 x_i}{\frac{1}{N} \sum_{j=1}^N w_2 w_1 x_j}$$

- Now, increasing w_2 or w_1 100x times will not change the partial derivative w.r.t. w_3 !
- The learning will become more stable

Batch normalization

[Szegedy and Ioffe 2015]



- Makes the training process invariant to some re-parameterizations
- Eliminates the bulk of cross-layer correlation between derivatives (off-diagonal Hessian vals)
- Use mini-batch statistics at training time to ensure that neuron activations are distributed “nicely” and the learning proceeds

Batch normalization layer

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_{1...m}\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

covariant to reparameterization → $\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i$ // mini-batch mean

→ $\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2$ // mini-batch variance

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{scale and shift}$$

learnable by SGD

[Szegedy and Ioffe 2015]

Batch normalization layer

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1 \dots x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

- At training time mean and variance are estimated per batch
- At test time, usually (running) averages over the dataset are used
- At test time, batch norm can be “merged in”
- For small batches, this is a big test $\leftrightarrow$ train mismatch ☹

Solutions to train-test mismatch:

- Keep training time behavior
- Switch to test behavior and fine-tune

Alternatives to BatchNorm

- Layer Norm [Ba et al. NIPS'16], Instance Norm [Ulyanov et al. Arxiv16], Group renorm [Wu and He, ECCV18] – normalize over statistics of certain specific groups of variables **within** the same sample
- Batch Renorm [Ioffe NIPS'17]: gradually switch between train and test time behavior during training
- Weight norm [Salimans and Kingma NIPS'16]: decouple direction and magnitude of weight matrices

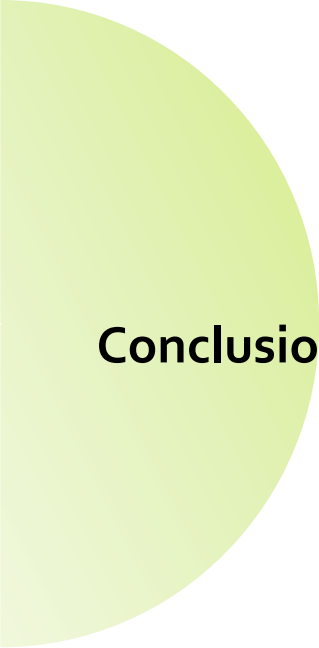
Initialization schemes

- **Basic idea 1:** units should be initialized to have comparable total input weights
- **Basic idea 2:** use layers which keep magnitude (otherwise both forwardprop and backprop will suffer from explosion/attenuation to zero; normalization layers solve this issue)
- E.g. [Glorot&Bengio 2010] aka “Xavier-initialization”:

$$W \sim U\left[-\frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}, \frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}\right]$$

- E.g. [He et al, Arxiv15] for ReLu networks:

$$W \sim \mathcal{N}(0, \sqrt{2/n_i})$$



Conclusions

Recap

- Batch SGD optimization is used in large-scale setting
- Advanced SGD methods use running averages to smooth and rescale SGD steps
- Normalization layers are important and used in most modern deep architectures

Bibliography

Léon Bottou, Olivier Bousquet:

The Tradeoffs of Large Scale Learning. NIPS 2007: 161-168

Nesterov, Yurii. "A method of solving a convex programming problem with convergence rate $O(1/k^2)$." Soviet Mathematics Doklady. Vol. 27. No. 2. 1983.

G. Goh, Why momentum really works? DISTILL 2017

<https://distill.pub/2017/momentum/>

John C. Duchi, Elad Hazan, Yoram Singer:

Adaptive Subgradient Methods for Online Learning and Stochastic Optimization. Journal of Machine Learning Research 12: 2121-2159 (2011)

Matthew D. Zeiler:

ADADELTA: An Adaptive Learning Rate Method. CoRR abs/1212.5701

Kingma, Diederik, and Jimmy Ba. "Adam: A method for stochastic optimization." ICLR 2015

Bibliography

Sergey Ioffe, Christian Szegedy:

Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.

ICML2015: 448-456

Sergey Ioffe, Batch Renormalization. NIPS 2017

Dmitry Ulyanov, Vadim Lebedev, Andrea Vedaldi, Victor S. Lempitsky:

Texture Networks: Feed-forward Synthesis of Textures and Stylized Images. ICML 2016: 1349-1357

Lei Jimmy Ba, Jamie Ryan Kiros, Geoffrey E. Hinton:

Layer Normalization. CoRR abs/1607.06450 (2016)

Yuxin Wu, Kaiming He: Group Normalization. ECCV (13) 2018: 3-19

Tim Salimans, Diederik P. Kingma:

Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks.

NIPS 2016: 901

Xavier Glorot, Yoshua Bengio:

Understanding the difficulty of training deep feedforward neural networks. AISTATS 2010: 249-256

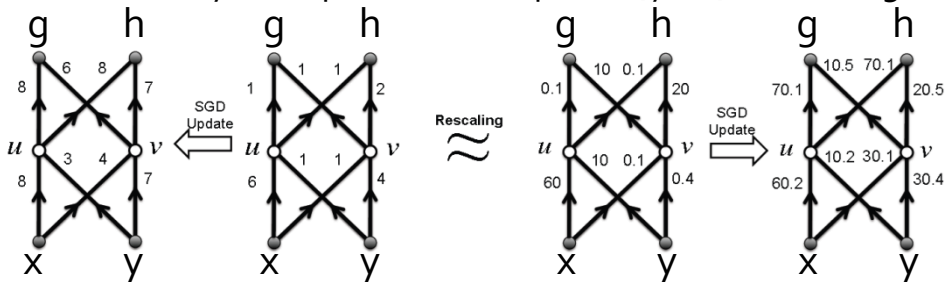
Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun:

Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. ICCV

2015: 1026-1034

Initialization schemes

One more toy example: 1 SGD step for $(x,y = 1,1)$ and $L = g+h$



[Neyshabur, Salakhutdinov, Srebro, Path-SGD: Path-Normalized Optimization in Deep Neural Networks, NIPS2015]

Units of measurements

- Let our coordinates be measured in meters. What is the unit of measurement for gradients?
Assume unitless function...
- (Stochastic) gradient descent is inconsistent.
- Newton method is consistent.



Adadelta method [Zeiler 2012]

$$g[t] = \mu g[t-1] + (1-\mu) \nabla(E, w[t]) \odot \nabla(E, w[t])$$

$$w[t+1] = w[t] - \frac{\sqrt{d[t]} + \epsilon}{\sqrt{g[t]} + \epsilon} \odot \nabla(E, w[t])$$

$$d[t+1] = \mu d[t] + (1-\mu) (w[t+1]-w[t]) \odot (w[t+1]-w[t])$$

- No step length parameter (good!)
- Correct units within the updates